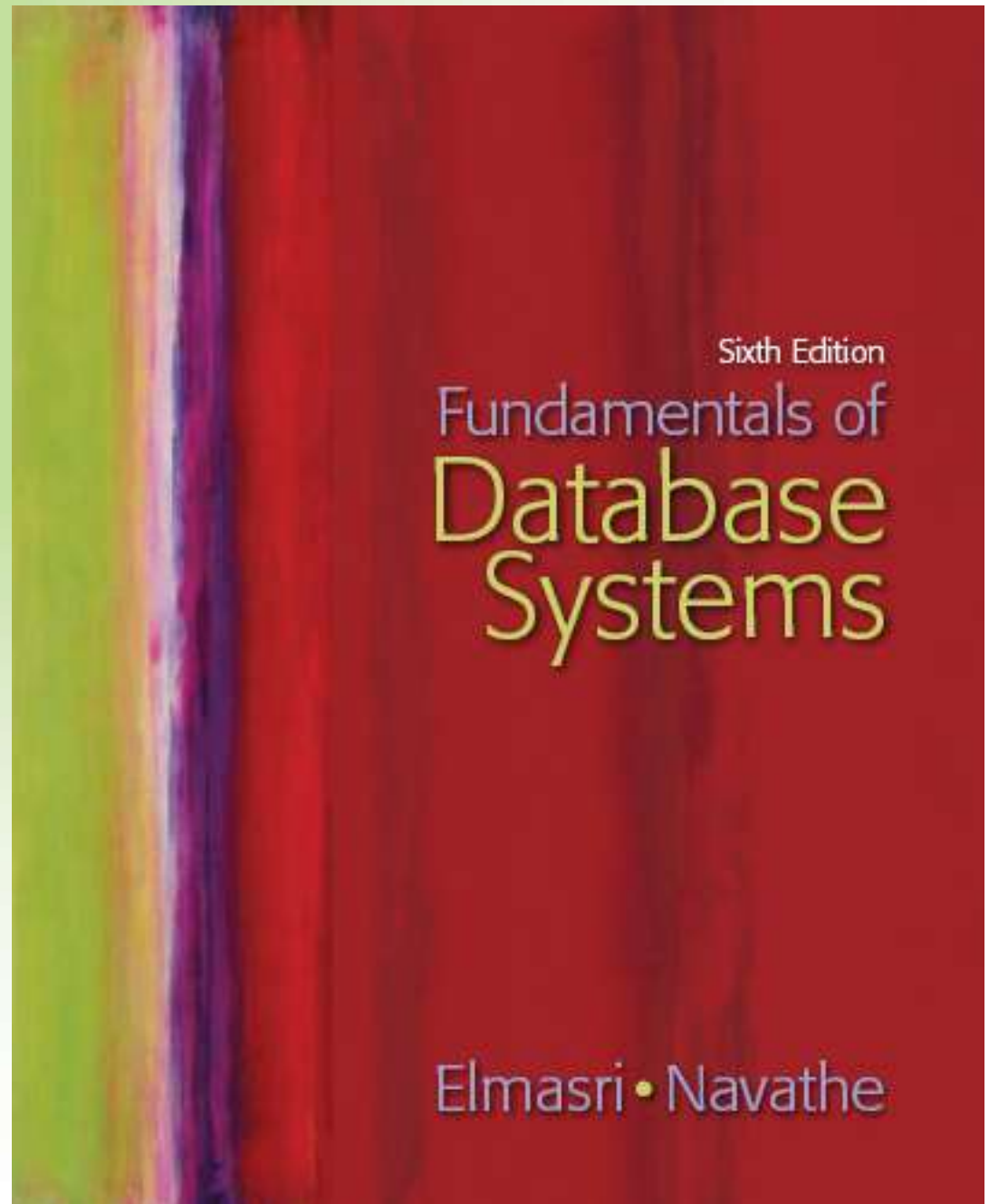


Chapter 11

Object and Object- Relational Databases



Addison-Wesley
is an imprint of

PEARSON

Object and Object-Relational Databases

- **Object databases (ODB)**
 - **Object data management systems (ODMS)**
 - Meet some of the needs of **more complex applications**
 - A database system that incorporates all the important object-oriented concepts
 - Specify:
 - Structure of **complex objects**
 - **Operations** that can be applied to these objects

Definition of an object

Objects – User defined complex data types

- An object has structure or state (variables) and methods (behavior/operations)

An object is described by four **basic** features

- Name: an object may also have a unique name in DB (**optional**)
- Identifier: a system-wide unique id for an object
- Lifetime: determines if the object is persistent or transient
- Structure: Construction of objects using type constructors

Characteristics of Object - Identifier

- Unique identity
 - Implemented via a unique, system-generated object identifier (OID)
 - **Immutable**
- Most OO database systems allow for the representation of both objects and literals (or values)

Characteristics of Object - Lifetime

- **Transient objects**

- Exist in executing program
- Disappear once program terminates

- **Persistent objects**

- Stored in database and persist after program termination
- **Naming mechanism**
- **Reachability**

Characteristics of Object - Structure

- Structure of arbitrary complexity
 - Contain all necessary information that describes object or literal
- Nesting **type constructors**
 - Construct complex type from other types
- Most basic constructors:
 - **Atom**
 - **Struct (or tuple)**
 - **Collection**

Complex Type Structures for Objects and Literals (cont'd.)

- Collection types:
 - **Set**
 - **Bag**
 - **List**
 - **Array**
 - **Dictionary**

Object-Oriented Database Architecture

There are two categories of object-oriented database architecture:

- Object-oriented database or standalone object-oriented database.
- Object-relational database.

Object-Oriented Database Architecture (Cont.)

Object-Oriented Database (OODB)

- Stores data as objects using object data model
- Directly saves program-created objects
- No conversion or mapping required
- Works naturally with OOP (classes & objects)
- Faster access than relational DBs

Object-Oriented Database Architecture (Cont.)

Object-Relational Database (ORDBMS)

- Maps object data into relational tables
- Acts as a staging/middle layer
- Converts objects for relational DB storage
- Supports applications using OOP + relational systems
- Useful when mixing object and relational models

Object-Oriented Advanced features

- Abstract Data Types
 - Class definition, provides extension to complex attribute types
- Encapsulation
 - Implementation of operations and object structure hidden
- Inheritance
 - Sharing of data within hierarchy scope, supports code reusability
- Polymorphism
 - Operator overloading

Object-Oriented Advanced features - Abstract Data Types

- Abstract Data Types
 - ADT = User-defined data type that encapsulates data + behavior.
 - Provides an extension to complex attribute types (beyond simple integers, strings, etc.).
 - Supports data abstraction → internal representation is hidden from the user.
 - Promotes modularity, reusability, and maintainability of code.

Object-Oriented Advanced features - Encapsulation

- Encapsulation

- Related to abstract data types and information hiding in programming languages
- Define **behavior** of a type of object based on operations that can be externally applied
- External users only aware of interface of the operations
- Divide structure of object into **visible and hidden attributes**

Object-Oriented Advanced features – Encapsulation (Cont.)

- **Object constructor**
 - Used to create a new object
- **Destructor** operation
 - Used to destroy (delete) an object
- **Modifier** operations
 - Modify the states (values) of various attributes of an object
- **Retrieve** information about the object
- Dot notation used to apply operations to object

Object-Oriented Advanced features - Inheritance

- Inheritance
 - Permits specification of new types or classes that inherit much of their structure and/or operations from previously defined types or classes
 - Definition of new types based on other predefined types
 - Leads to **class hierarchy**

Type Hierarchies and Inheritance (cont'd.)

- **Selective inheritance**

- Subtype inherits only some of the functions of a supertype

- **Multiple inheritance**

- Subtype inherits functions (attributes and methods) of more than one supertype

Object-Oriented Advanced features – **Polymorphism**

- **Polymorphism** of operations
 - Also known as **operator overloading**
 - Allows same operator name or symbol to be bound to two or more different implementations
 - Depending on type of objects to which operator is applied

The ODMG Object Model and the Object Definition Language ODL

- ODMG object model (Object Data Management Group Model)
 - is a **standard model for object-oriented databases (OODBMS)** developed by the **Object Data Management Group (ODMG)** in the early 1990s.
 - **Components for ODMG** - **object definition language (ODL)** and **object query language (OQL)**

Object Definition Language (ODL)

- Object Definition Language ODL
 - Support semantic constructs of ODMG object model
 - Independent of any particular programming language
- **Behavior** refers to operations
- **State** refers to properties
- **Class**
 - Specifies both state (attributes) and behavior (operations) of an object type
 - A **class** defines a **concrete object type** that can have **actual instances** stored in the database.
 - A class can **implement one or more interfaces**, meaning it provides the actual data and operations that the interface describes.
 - **Instantiable**

Object Definition Language ODL

- Atomic (User-Defined) Objects

- Specified using keyword **class** in ODL
- **Attribute**
 - Property; describes some aspect of an object
- **Relationship**
 - Two objects in the database are related
 - Keyword **inverse**
 - Single conceptual **relationship** in **bidirectional directions**
- **Operation signature:**
 - Operation name, argument types, return value

Object Definition Language ODL

- Built-in Interfaces and Classes

- **Collection objects**

- Inherit the basic Collection interface
- **Interface = Object Type / Class Definition** in the database schema.

- `I = O.create_iterator()`

- Creates an iterator object for the collection

- Collection objects further specialized into:

- `set, list, bag, array, and dictionary`

- **Extent**

- Store **collection** of **persistent objects for each type** or subtype
- Extents are subsets of the extent of class OBJECT

Object Definition Language ODL

- Built-in Interfaces and Classes (cont'd.)

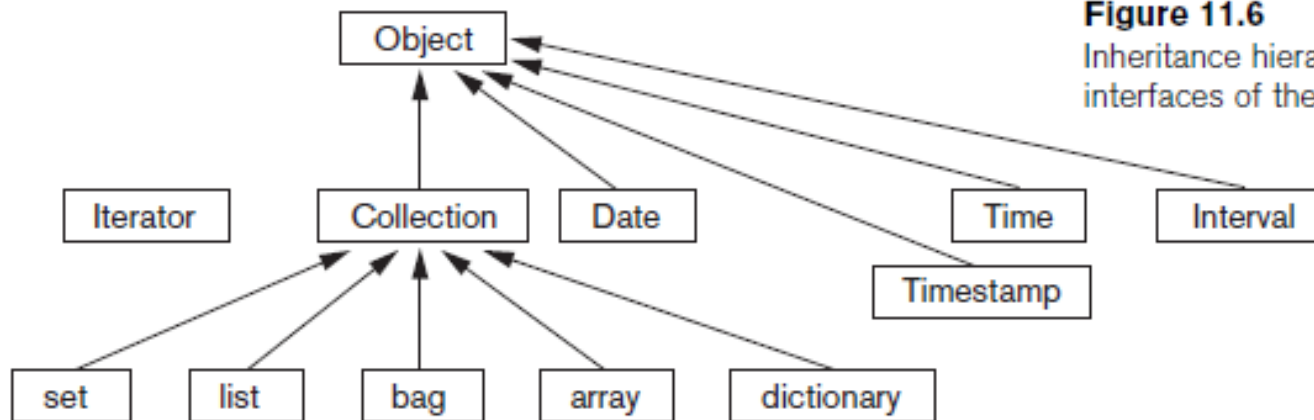


Figure 11.6

Inheritance hierarchy for the built-in interfaces of the object model.

Object Definition Language ODL

- Inheritance in the Object Model

- **EXTENDS inheritance**

- Specified by keyword **extends**
- Inherit both state and behavior strictly among classes
- Multiple inheritance via extends not permitted

Figure 11.10

Possible ODL schema for the UNIVERSITY database in Figure 11.8(b).

```

class PERSON
(
  extent    PERSONS
  key       Ssn )
{
  attribute  struct Pname {   string  Fname,
                              string  Mname,
                              string  Lname }   Name;

  attribute  string           Ssn;
  attribute  date             Birth_date;
  attribute  enum Gender{M, F} Sex;
  attribute  struct Address {  short   No,
                              string  Street,
                              short   Apt_no,
                              string  City,
                              string  State,
                              short   Zip }   Address;

  short      Age(); };

class FACULTY extends PERSON
(
  extent    FACULTY )
{
  attribute  string           Rank;
  attribute  float            Salary;
  attribute  string           Office;
  attribute  string           Phone;
  relationship DEPARTMENT    Works_in inverse DEPARTMENT::Has faculty;
  relationship set<GRAD_STUDENT> Advises inverse GRAD_STUDENT::Advisor;
  relationship set<GRAD_STUDENT> On_committee_of inverse GRAD_STUDENT::Committee;
  void       give_raise(in float raise);
  void       promote(in string new rank); };

class GRADE
(
  extent    GRADES )
{
  attribute  enum GradeValues{A,B,C,D,F,I, P} Grade;
  relationship SECTION Section inverse SECTION::Students;
  relationship STUDENT Student inverse STUDENT::Completed_sections; };

```


The Object Query Language OQL

- Query language proposed for ODMG object model
- Declarative query language
 - Not computationally complete
- Syntax based on SQL (select, from, where)
- Additional flexibility (queries with user defined operators and types)

Object Query Language (OQL)

- Simple OQL queries, database entry points, and iterator variables
 - Syntax: select ... from ... where ... structure
 - Entry point: named persistent object
 - Iterator variable: define whenever a collection is referenced in an OQL query

Features of OQL

- **Named query**
 - Specify identifier of named query
- OQL query will **return collection** as its result
 - If user requires that a query only return a **single element** use **element** operator
- **Aggregate operators**

Other Features of OQL (cont'd.)

- Special operations for ordered collections
- **Group by** clause in OQL
 - Similar to the corresponding clause in SQL
 - Provides explicit reference to the **collection of objects within each group** or **partition**
- **Having** clause
 - Used to filter partitioned sets

Example of OQL query

The following is a sample query

“what are the names of the black product?”

Select distinct p.name

From products p

Where p.color = “black”

⇒ Valid in both SQL and OQL, but results are different.

Result of the query (SQL)

Original table

Product no	Name	Color
P1	Ford Mustang	Black
P2	Toyota Celica	Green
P3	Mercedes SLK	Black

Result

Name
Ford Mustang
Mercedes SLK

- The statement queries a relational database.

=> Returns a table with rows.

Result of the query (OQL)

Original table

Product no	Name	Color
P1	Ford Mustang	Black
P2	Toyota Celica	Green
P3	Mercedes SLK	Black

Result

String	String
Ford Mustang	Mercedes SLK

- The statement queries a
object-oriented database
=> Returns a collection of
objects.

Comparison

- Queries look very similar in SQL and OQL, sometimes they are the same
- In fact, the results they give are very different

Query returns:

OQL	SQL
Object	Tuple
Collection of objects	Table

SQL3 “Object-oriented SQL”

- Foundation for several OO database management systems – ORACLE8, DB2, etc
- New features – “relational” & “Object oriented”
- Relational Features – new data types, new predicates, enhanced semantics, additional security and an active database
- Object Oriented Features – support for functions and procedures

User defined Data Types

Creating a “row type”

- A **user-defined object structure** that specifies the attributes of a “row.”
- type = **blueprint** for a single object / tuple.

Example:

```
create row type AddressType(  
    street    char(50),  
    city      char(20));
```

```
create row type StarType(  
    name      char(30),  
    address AddressType);
```

Creating Data Types (contd.)

Creating “Table”

- A **collection of rows** (tuples) of a given row type.
- Table = **set of objects** following the blueprint.

create table Address **of type** AddressType;

create table MovieStar **of type** StarType;

Instances of Row types are tuples in tables

Sample Query

Find the names and street addresses of those MovieStars who stay in the city “Columbus”:

```
select MovieStar.name,  
        MovieStar.address.street  
from   MovieStar  
where MovieStar.address.city = “Columbus”;
```

Creating Functions

```
create function one() returns int  
as 'select 1 as RESULT'  
language 'sql';
```

//It returns an integer

//If you write language 'sql', PostgreSQL knows the body is a plain SQL query.

//If you write language 'plpgsql', PostgreSQL expects **procedural code** with **BEGIN ... END;**

```
select one() as answer;
```

answer
1

Creating “tables” with “methods”

Implementation

```
create table slides (  
    id      int,  
    date    date,  
    caption document,  
    picture CD_image,  
    method containsName  
        (name varchar)  
    returns boolean );  
  
create table landmarks(  
    name      varchar (30),  
    location   point);
```

Implementation (contd.)

Sample query – find a picture of a reservoir with low water level which is in “Sacramento”

```
select P.id  
from    slides P, landmarks L  
where IsLowWaterLevel (P.picture) and  
P.containsName (L.name) and          L.name =  
“Sacramento”;
```